Complete Guide to Windows Subsystem for Linux

https://gemini.google.com/app/8ca0165176a33604

User prompt: Windows Subsystem for Linux

Response: **Windows Subsystem for Linux (WSL)** is a feature of Windows that allows developers to run a GNU/Linux environment—including command-line tools, utilities, and applications—directly on Windows, unmodified, without the overhead of a traditional virtual machine or dual-boot setup.

Core Architecture: WSL 1 vs. WSL 2

Feature	WSL 1	WSL 2
Architecture	Translation layer (maps Linux system calls to Windows NT kernel)	Full Linux kernel running in a lightweight Hyper-V utility VM
File I/O Performance	Slower across Linux root; fast across `/mnt/c/`	**Fast native Linux file system** performance; slower across `/mnt/c/`
System Call Compatibility	Partial (limited low-level/network socket support)	**100% full system call compatibility**
Docker & Containers	Limited / Workarounds required	**Native support** (runs Docker Desktop engine directly)
GPU Acceleration	Not supported	**Supported** (CUDA, Direct3D 12, PyTorch/TensorFlow)
GUI Support (WSLg)	Third-party X-servers required	**Native integration** (Wayland/X11 app rendering out-of-the-box)

Key Capabilities

- **Linux Distributions:** Install multiple distributions side-by-side from the Microsoft Store or command line (e.g., Ubuntu, Debian, Arch Linux, openSUSE, AlmaLinux).
- **Cross-OS Interoperability:**
  - Invoke Windows executables from the Linux bash terminal (e.g., `cmd.exe /c dir` or `explorer.exe .`).
  - Run Linux commands directly from Windows PowerShell or Command Prompt (`wsl ls -la`).
  - Access Windows files from Linux (`/mnt/c/Users/...`) and access Linux files from Windows Explorer via `\\wsl$\`.

- **Hardware & GPU Passthrough:** Direct access to NVIDIA CUDA, AMD ROCm, and Direct3D 12 for high-performance computing, machine learning, and graphics workflows.
- **WSLg (GUI Applications):** Enables running graphical Linux tools (such as GIMP, VS Code Linux build, or Qt applications) seamlessly as native Windows windows with audio and hardware graphics acceleration.

Essential WSL Management Commands

Execute these commands in Windows PowerShell or Command Prompt:

```
# Install default Linux distribution (Ubuntu)
wsl --install

# List available online distributions
wsl --list --online

# Install a specific distribution
wsl --install -d Ubuntu-24.04

# List installed distributions and their current WSL version
wsl --list --verbose

# Set default version for future installations
wsl --set-default-version 2

# Upgrade an existing distribution to WSL 2
wsl --set-version <DistroName> 2

# Terminate a running distribution
wsl --terminate <DistroName>

# Shut down the entire WSL VM instance
wsl --shutdown
```

Recommended Workflows & Configuration

1. **File Storage Location:** To achieve maximum I/O performance in WSL 2, store source code and active project files within the Linux root filesystem (e.g., `~/projects/`) rather than the mounted Windows drive (`/mnt/c/`).
2. **Configuration File (`.wslconfig`):** Tune resource allocation by creating or editing `%USERPROFILE%\.wslconfig` in Windows:

   ```
   [wsl2]
   memory=8GB        # Limits memory allocated to the WSL 2 VM
   processors=4      # Sets available virtual processors
   swap=4GB          # Configures swap space
   localhostForwarding=true
   ```

3. **Enable `systemd`:** To run system background services natively (e.g., `systemctl start docker`), add the following to `/etc/wsl.conf` inside your Linux distro:

   ```
   [boot]
   systemd=true
   ```
